

ИНЖЕНЕРИЯ AI-АГЕНТОВ

Теоретический путеводитель для поездки

От языковой модели и tool calling до harness engineering, evals, RAG, MCP, наблюдаемости и безопасного production-запуска.

Для Кирилла · версия от 16 июля 2026 года

Содержание

1. Как пользоваться книгой
2. Глава 1. Кто такой AI Engineer
3. Глава 2. Как устроена LLM
4. Глава 3. Structured Outputs и Tool Calling
5. Глава 4. Workflow, агент и уровни автономности
6. Глава 5. Agent Harness и Harness Engineering
7. Глава 6. State, Memory и Context Engineering
8. Глава 7. RAG и работа со знаниями
9. Глава 8. MCP и интеграции
10. Глава 9. Планирование, Reflection и Multi-agent
11. Глава 10. Evals и Observability
12. Глава 11. Self-improving Agents
13. Глава 12. Production: безопасность, стоимость и эксплуатация
14. Глава 13. Карта фреймворков
15. Глава 14. SoloGPT и «Колли» как полигон
16. Глава 15. Как проектировать нового агента
17. Глава 16. План чтения до 1 августа
18. Краткий словарь
19. Источники для дальнейшего чтения

Как пользоваться книгой

Эта книга — введение в профессию, а не инструкция по одной библиотеке. Её задача — дать общую карту: что такое современный AI-продукт, где в нём находится агент, как инженер делает его надёжным и как проверяет качество.

Не старайся запомнить все английские термины с первого раза. После каждой главы ответь на вопросы своими словами. Если можешь объяснить понятие без определения из книги — значит, начал его понимать.

Во время поездки не нужен компьютер. Полезно иметь заметки в телефоне и для каждого придуманного агента записывать семь пунктов:

1. Какую бизнес-задачу он решает?
2. Что поступает ему на вход?
3. Какие действия ему разрешены?
4. Где хранится состояние?
5. Как он понимает, что закончил?
6. Что может пойти не так?
7. Как измерить качество?

Главный принцип всей книги:

AI-агент — не магический разум. Это вероятностная модель внутри спроектированной инженером системы.

Глава 1. Кто такой AI Engineer

AI Engineer создаёт программные продукты, в которых языковая модель является одним из компонентов. Он не только пишет prompt и вызывает API. Он проектирует весь путь данных: от входного события до результата, действия, проверки и наблюдения после запуска.

В обычном backend-приложении большинство правил задаёт программист. Если пользователь отправил корректный запрос, код выполняет заранее известную последовательность операций. Результат должен быть одинаковым при одинаковых условиях.

LLM работает иначе. Она выбирает вероятный ответ. Даже хороший prompt не превращает её в обычную функцию. Ответ может быть полезным, неполным, неожиданным или неправильным. Поэтому AI Engineer совмещает две культуры:

- классическую разработку, где важны типы, тесты, API, базы данных и надёжность;
- работу с вероятностной моделью, где важны контекст, evals, ограничения и наблюдение за поведением.

Что делает Agent Engineer

Agent Engineer специализируется на системах, где модель может выполнять несколько шагов, выбирать инструменты и менять состояние внешней среды.

Например, агент получает задачу изучить клиента. Он может:

1. запросить сведения из CRM;
2. проверить платежи;
3. найти последние обращения в поддержку;

4. сопоставить факты с правилами продукта;
5. сформировать рекомендацию;
6. создать задачу менеджеру;
7. сохранить причину своего решения.

Здесь модель уже не просто пишет текст. Она участвует в управлении процессом.

Почему инженерная база остаётся главной

Агент взаимодействует с реальным миром через обычный код. CRM, платежи, документы, очереди задач, авторизация, API и базы данных не исчезают из-за появления LLM.

Поэтому профессия требует:

- Python и структур данных;
- HTTP, API и FastAPI;
- SQL;
- async и фоновых задач;
- Git, Docker и CI/CD;
- тестирования;
- логирования и безопасности;
- умения разбирать требования бизнеса.

LLM добавляет новый слой, но не заменяет остальные.

Самопроверка

- Чем AI Engineer отличается от человека, который умеет пользоваться ChatGPT?
- Почему хороший prompt не заменяет backend?

- Какие последствия будут, если дать модели прямой доступ к CRM без проверок?

Глава 2. Как устроена LLM

LLM — большая языковая модель. Упрощённо она получает последовательность токенов и предсказывает продолжение.

Токен — это небольшой фрагмент текста. Он может быть словом, частью слова, знаком или числом. Модель не читает предложение как человек. Она преобразует токены в числовые представления и на их основе вычисляет вероятности следующих токенов.

Что модель знает и чего не знает

Во время обучения модель увидела большой объём текстов и научилась находить закономерности. Это позволяет ей объяснять, классифицировать, обобщать, писать код и рассуждать.

Но у неё нет автоматического доступа к:

- текущей базе клиентов;
- актуальной цене продукта;
- внутренним документам компании;
- результату только что выполненного API-запроса;
- намерениям пользователя, которые не выражены в контексте.

Такие сведения должна предоставить система.

Контекстное окно

Контекстное окно — объём информации, доступный модели в одном запуске. Туда входят инструкции, сообщения, документы, результаты tools и иногда рассуждения предыдущих шагов.

Большое окно не означает, что нужно складывать в него всё. Лишний контекст:

- повышает стоимость;
- увеличивает задержку;
- отвлекает модель;
- затрудняет поиск важного факта;
- может содержать конфликтующие инструкции.

Context engineering — проектирование того, какая информация, в какой форме и в какой момент попадёт модели.

Инструкции и данные

Полезно разделять:

- постоянную роль и правила агента;
- конкретную задачу пользователя;
- факты из бизнес-систем;
- найденные документы;
- результаты инструментов.

Если смешать всё в один длинный текст, станет трудно понять источник ошибки.

Температура и недетерминированность

Параметры генерации могут влиять на разнообразие ответа, но низкая температура не превращает модель в гарантированно детерминированную программу. Для строгих требований нужны схемы, кодовая валидация и evals.

Самопроверка

- Почему модель может уверенно сообщить неправильный факт?
- Чем контекст отличается от знаний, полученных при обучении?

- Почему «положить всю базу в prompt» — плохая архитектура?

Глава 3. Structured Outputs и Tool Calling

Текст удобен человеку, но неудобен программе. Если модель отвечает: «Наверное, стоит связаться с клиентом на следующей неделе», backend не знает точного приоритета, срока и вида действия.

Structured Output — ответ по заранее заданной структуре.

Пример:

```
{
  "action": "contact_client",
  "priority": "high",
  "reason": "subscription_expires_soon",
  "days_until_deadline": 3
}
```

Схема определяет обязательные поля, типы и допустимые значения. После ответа код всё равно обязан проверить данные.

Function Calling

Function Calling, или Tool Calling, позволяет модели попросить программу вызвать инструмент.

Модель не выполняет Python-функцию сама. Процесс выглядит так:

1. приложение описывает модели доступные tools;
2. модель выбирает tool и формирует аргументы;
3. приложение проверяет аргументы и права;
4. обычный код выполняет действие;
5. результат возвращается модели;
6. модель решает, что делать дальше.

Tool может читать данные или изменять мир.

Примеры чтения:

- получить карточку клиента;
- найти документ;
- проверить статус заказа.

Примеры изменения:

- создать задачу;
- отправить письмо;
- изменить тариф;
- вернуть деньги.

Вторая группа требует более строгих прав и часто подтверждения человеком.

Хороший инструмент

Хороший tool:

- решает одну понятную задачу;
- имеет короткое точное описание;
- принимает строгую схему аргументов;
- возвращает полезный структурированный результат;
- сообщает понятные ошибки;
- не даёт больше прав, чем необходимо;
- допускает повторный безопасный вызов или защищён от дублей.

Последнее свойство называется идемпотентностью. Например, повтор webhook не должен создать вторую одинаковую задачу менеджеру.

Самопроверка

- Кто фактически выполняет tool: модель или приложение?
- Почему `execute_sql(query: str)` опаснее, чем `get_client(client_id: str)`?
- Какие действия в твоём будущем агенте должны требовать подтверждения?

Глава 4. Workflow, агент и уровни автономности

Не всякая программа с LLM является агентом.

Один вызов модели

Вход отправляется модели, ответ возвращается пользователю. Модель не выбирает инструменты и не выполняет последовательность шагов.

Пример: кратко пересказать документ.

Workflow

Последовательность определяет программист.

извлечь данные → классифицировать → сформировать текст → проверить схему

LLM может участвовать в отдельных шагах, но маршрут известен заранее.

Workflow подходит, если процесс стабилен и предсказуем. Он проще тестируется, дешевле работает и легче объясняется.

Agent Loop

Модель сама выбирает следующий tool на основе цели и предыдущего результата.

```

цель
↓
модель выбирает действие
↓
tool изменяет или наблюдает среду
↓
результат возвращается модели
↓
новое решение или завершение
```

Агент подходит для открытых задач, где заранее неизвестно точное количество шагов.

Лестница автономности

1. Один LLM-вызов.
2. Workflow с фиксированными шагами.
3. Agent loop с выбором tools.
4. Stateful agent, сохраняющий состояние.
5. Long-running agent с checkpoints и продолжением работы.
6. Multi-agent система.
7. Контролируемый self-improving loop.

Не нужно начинать с верхней ступени. Каждый уровень добавляет новые способы ошибиться.

Stopping Conditions

Агент должен понимать, когда остановиться. Ограничения могут включать:

- максимальное количество шагов;
- максимальное время;
- лимит стоимости;
- список разрешённых действий;
- достижение проверяемого результата;
- обязательную остановку перед рискованным tool;
- остановку после повторяющихся ошибок.

Без stopping conditions агент способен заиклиться, потратить бюджет или многократно выполнить действие.

Самопроверка

- Для какой задачи лучше workflow, а не агент?
- Почему больше автономности не всегда означает лучший продукт?
- Какие stopping conditions нужны исследовательскому агенту?

Глава 5. Agent Harness и Harness Engineering

Модель сама по себе редко является готовым агентом. Ей нужна инженерная среда — harness.

Harness можно представить как рабочее место сотрудника. Даже сильный специалист работает хуже, если у него нет доступа к данным, понятной задаче, инструментов, правил безопасности и способа передать результат.

Компоненты harness

Инструкции

Описывают цель, ограничения, доступные действия и формат результата.

Контекст

Факты, необходимые прямо сейчас: данные пользователя, документы, история и результаты tools.

Tools

Безопасные способы наблюдать среду и выполнять действия.

Runtime

Код, который вызывает модель, исполняет tools, возвращает результаты и управляет циклом.

State и Memory

Состояние текущей работы и данные, которые должны пережить один вызов модели.

Checkpoints

Снимки прогресса, позволяющие остановить и продолжить длинную задачу.

Permissions и Sandbox

Ограничивают доступ. Sandbox — изолированная среда, где ошибка агента не должна повредить production.

Guardrails

Проверки входа, результата или tool call. Они не гарантируют абсолютную безопасность, но уменьшают риск.

Traces и Evals

Trace показывает путь выполнения. Eval измеряет качество на наборе сценариев.

Длинные задачи

Контекст модели ограничен, поэтому long-running agent должен оставлять артефакты:

- план;
- список выполненного;
- текущий статус;
- промежуточные файлы;
- найденные проблемы;
- следующий маленький шаг.

Новый запуск читает эти артефакты и продолжает работу. Это похоже на передачу смены между инженерами.

Почему harness важнее выбора модели

Более сильная модель может временно повысить качество. Но плохие tools, отсутствующие ограничения и слабые evals сохраняют системные ошибки. Хороший harness позволяет менять модель и сравнивать результат, не переписывая весь продукт.

Самопроверка

- Какие части harness универсальны для разных агентов?
- Что должен сохранить агент перед завершением контекстного окна?
- Чем guardrail отличается от eval?

Глава 6. State, Memory и Context Engineering

Термины state, memory и context часто смешивают.

State

State — текущее состояние процесса.

Например:

```
task_id: 145
status: waiting_for_approval
completed_steps: [load_client, analyze_payments]
proposed_action: upgrade_plan
```

State должен храниться надёжно и быть доступен после перезапуска сервиса.

Краткосрочная память

Информация, нужная внутри текущего диалога или запуска: последние сообщения, план и результаты tools.

Долгосрочная память

Информация между запусками: предпочтения пользователя, подтверждённые факты, история решений или полученная обратная связь.

Не каждое сообщение нужно превращать в долгосрочную память. Ошибка, случайная фраза или устаревший факт могут загрязнить будущий контекст.

Context Engineering

Задача инженера — выбрать минимальный полезный контекст.

Типичный процесс:

1. определить цель текущего шага;
2. получить нужные структурированные данные;
3. найти релевантные документы;
4. сжать длинную историю;
5. удалить дубли и устаревшее;
6. обозначить источники и доверие к ним;
7. собрать контекст в понятные секции.

Compaction

Compaction — сжатие истории или промежуточных результатов.

Хорошее сжатие сохраняет решения, факты, ограничения и незавершённые задачи. Плохое — оставляет красивый пересказ, но теряет сведения, необходимые для продолжения.

Самопроверка

- Чем state отличается от текста предыдущего сообщения?
- Какие сведения нельзя автоматически записывать в долгосрочную память?
- Что обязательно сохранить при сжатии длинной задачи?

Глава 7. RAG и работа со знаниями

RAG расшифровывается как Retrieval-Augmented Generation — генерация, дополненная поиском.

Модель не должна отвечать по памяти, если компании нужен ответ из актуальных документов. Сначала система ищет подходящие фрагменты, затем передаёт их модели.

Основные этапы RAG

Ingestion

Документы загружаются, очищаются и делятся на фрагменты — chunks.

Embeddings

Фрагменты превращаются в числовые векторы, отражающие смысловую близость.

Retrieval

По запросу выбираются наиболее похожие фрагменты.

Reranking

Кандидаты дополнительно сортируются более точным способом.

Generation

Модель формирует ответ на основе найденного контекста и указывает источники.

Почему RAG может ошибаться

- нужный документ не загружен;

- документ устарел;
- chunks разделены неудачно;
- поиск выбрал похожий, но нерелевантный текст;
- модель проигнорировала источник;
- документ содержит вредоносную инструкцию;
- правильного ответа в базе нет.

Поэтому отдельно оценивают:

- retrieval: нашёлся ли правильный фрагмент;
- groundedness: основан ли ответ на найденном тексте;
- answer quality: полезен ли итог пользователю;
- abstention: умеет ли система честно отказаться.

RAG и агент

RAG может быть одним фиксированным шагом workflow. Агентный RAG позволяет модели самой сформировать поисковый запрос, выполнить несколько поисков, уточнить вопрос и решить, достаточно ли информации.

Вторая архитектура гибче, но дороже и сложнее для оценки.

Самопроверка

- Почему наличие vector database не гарантирует хороший RAG?
- Какие три части RAG нужно оценивать отдельно?
- Когда один поиск лучше агентного исследования?

Глава 8. MCP и интеграции

MCP — Model Context Protocol. Это стандарт взаимодействия AI-приложения с источниками данных и инструментами.

MCP не является моделью и не делает систему агентом автоматически.

Роли

MCP Host

Приложение, в котором работает пользователь и модель.

MCP Client

Компонент host, который подключается к конкретному MCP server.

MCP Server

Предоставляет возможности через стандартный протокол.

Основные primitives

- **Tools** — действия и запросы, которые может вызвать модель.
- **Resources** — данные, которые приложение может добавить в контекст.
- **Prompts** — переиспользуемые шаблоны взаимодействия.

Например, CRM MCP Server может предоставить:

- tool `get_client`;
- tool `create_task`;
- resource со схемой CRM;
- prompt с правилами подготовки отчёта.

Зачем нужен MCP

Без стандарта для каждого AI-клиента пришлось бы заново писать интеграцию. MCP отделяет бизнес-возможности от конкретной модели или фреймворка.

Но стандарт не решает автоматически:

- авторизацию;
- бизнес-права;
- подтверждение опасных действий;
- качество описания tools;
- защиту данных;
- обработку ошибок.

Эти обязанности остаются у инженера.

Самопроверка

- Чем MCP отличается от REST API?
- Почему MCP server не обязан содержать LLM?
- Какие права должен иметь CRM MCP Server для учебного агента?

Глава 9. Планирование, Reflection и Multi-agent

Планирование

План помогает разбить большую цель на шаги. Он полезен для исследований, разработки и работы с несколькими источниками.

Но план не должен быть неизменным. После результата tool агент может обнаружить, что исходное предположение ошибочно. Тогда план обновляется.

Полезный план содержит:

- маленькие проверяемые шаги;
- зависимости;
- критерий завершения;
- текущий статус;
- ограничения времени и стоимости.

Reflection и Self-critique

Reflection — дополнительный шаг, на котором модель проверяет собственный результат и предлагает исправление.

Это может улучшить ответ, но модель способна не заметить собственную ошибку или уверенно одобрить неправильный результат. Self-critique не заменяет независимую проверку.

Надёжнее сочетать:

- кодовые проверки;
- источники ground truth;

- отдельный evaluator;
- человеческую оценку;
- regression-набор.

Multi-agent

Multi-agent система использует несколько агентов с разными ролями.

Пример исследования:

- ведущий агент строит план;
- subagents параллельно изучают разные источники;
- ведущий объединяет результаты;
- evaluator проверяет полноту.

Преимущества:

- параллельность;
- специализация;
- разделение контекста;
- независимая проверка.

Недостатки:

- выше стоимость;
- сложнее координация;
- возможны дубли и конфликты;
- труднее понять источник ошибки;
- сложнее evals.

Сначала нужно построить хороший single-agent baseline. Multi-agent добавляется только после измеримого сравнения.

Самопроверка

- Почему первоначальный план агента может измениться?
- Почему self-critique не является гарантией правильности?
- В каком сценарии несколько агентов дадут реальное преимущество?

Глава 10. Evals и Observability

Обычный unit-тест проверяет детерминированный код: для заданного входа ожидается точный результат.

LLM может дать несколько разных, но приемлемых ответов. Поэтому появляются evals — систематическая оценка качества на наборе примеров.

Что можно оценивать

- финальный результат;
- соблюдение структуры;
- правильность выбора tool;
- корректность аргументов;
- траекторию действий;
- использование источников;
- безопасность;
- число шагов;
- стоимость;
- задержку;
- успешность бизнес-действия.

Виды проверок

Кодовые

Проверяют схему, числа, допустимые действия, наличие источников и лимиты.

Эталонные примеры

Сравнивают с заранее подготовленным правильным результатом или допустимыми критериями.

Human Review

Эксперт оценивает полезность, корректность и стиль.

LLM-as-Judge

Другая модель оценивает результат по рубрике. Это масштабируемо, но evaluator тоже может ошибаться и иметь систематическое смещение.

Offline и Online Evals

Offline evals запускаются до релиза на фиксированном наборе. Они сравнивают старую и новую версию и ловят регрессии.

Online evals и monitoring работают на production traces. Они ищут новые типы ошибок, изменения качества и необычные сценарии.

Неудачные production-примеры после очистки персональных данных могут пополнять offline-набор.

Observability

Observability — способность понять внутреннее состояние системы по внешним данным.

Для агента полезны:

- trace всего запуска;
- spans отдельных model и tool calls;
- версия prompt, модели и tools;
- токены и стоимость;

- latency;
- ошибки и retries;
- решение human-in-the-loop;
- итоговая обратная связь.

Если агент ошибся, инженер должен восстановить не только ответ, но и путь к нему.

Самопроверка

- Почему unit-тестов недостаточно для агента?
- Чем trace отличается от обычного финального лога?
- Как production-ошибка превращается в regression test?

Глава 11. Self-improving Agents

Фраза «самообучающийся агент» может означать разные вещи.

Уровень 1. Исправление текущего результата

Агент проверяет ответ и повторяет попытку. Изменение не сохраняется между запусками.

Уровень 2. Использование обратной связи

Система сохраняет оценку человека и использует её как контекст или пример в будущем.

Уровень 3. Предложение изменения

Агент анализирует группу ошибок и предлагает изменить prompt, skill, tool description или retrieval.

Уровень 4. Автоматический эксперимент

Предложенная версия запускается в sandbox на eval-наборе и сравнивается с baseline.

Уровень 5. Контролируемый релиз

Человек изучает результаты, риски и регрессии и одобряет новую версию.

Уровень 6. Полностью самостоятельное изменение production

Агент сам меняет код, правила или права и выпускает себя. Это высокий риск, особенно в системах с деньгами, персональными данными или клиентскими коммуникациями.

В нашем курсе self-improving loop выглядит так:

traces и обратная связь
↓
группировка повторяющихся ошибок
↓
предложение новой версии
↓
sandbox + offline evals
↓
сравнение с baseline
↓
одобрение человеком
↓
контролируемый релиз и monitoring

Что может улучшаться

- prompt;
- примеры;
- skill или инструкция;
- описание tool;
- порядок шагов workflow;
- retrieval и chunks;
- выбор модели;
- критерий handoff;
- eval-набор.

Важно не позволять системе улучшать только метрику, забывая о реальной цели. Если evaluator слабый, агент может научиться получать высокий балл, не становясь полезнее.

Самопроверка

- Какие уровни self-improvement приемлемы для production?
- Почему нельзя позволять агенту изменять одновременно prompt и eval?

- Что такое baseline и зачем он нужен?

Глава 12. Production:

безопасность, стоимость и эксплуатация

Демонстрация работает на нескольких примерах. Production должен работать долго, с реальными ошибками и ограничениями.

Надёжность

Нужно предусмотреть:

- сетевые ошибки;
- rate limits;
- таймауты;
- недоступность базы;
- неправильный tool result;
- повтор события;
- частично выполненное действие;
- перезапуск worker;
- закливание агента.

Безопасность

Принцип минимальных прав: агент получает только те возможности, которые нужны для задачи.

Чтение и изменение разделяются. Опасные действия требуют подтверждения. Секреты не передаются модели без необходимости и не записываются в Git или traces.

Prompt injection — попытка внедрить инструкцию через пользовательский текст или документ. Нельзя считать внешний документ доверенной системной инструкцией.

Стоимость

Стоимость зависит от:

- выбранной модели;
- количества входных и выходных токенов;
- числа шагов;
- повторных попыток;
- retrieval и внешних сервисов;
- параллельных subagents;
- хранения и observability.

Инженер считает стоимость не одного красивого ответа, а успешного выполнения бизнес-задачи.

Задержка

Agent loop может вызвать модель и tools несколько раз.

Пользователь не должен ждать без информации. Применяются streaming, фоновые задачи, статусы выполнения и уведомления.

Деплой

Типичная система включает:

- FastAPI для запросов;
- worker для долгих задач;
- SQL-базу для состояния;
- очередь;

- хранилище документов;
- secrets manager или переменные окружения;
- Docker;
- CI/CD;
- monitoring и alerting.

Не все компоненты нужны в первом проекте. Архитектура растёт вместе с нагрузкой и риском.

Самопроверка

- Почему агенту нельзя выдавать административный API-ключ?
- Как защититься от повторного выполнения действия?
- Какие данные нужны для расчёта стоимости одного успешного запуска?

Глава 13. Карта фреймворков

Фреймворк ускоряет разработку, но может скрыть механизм. Поэтому сначала полезно понять agent loop на обычном Python.

Чистый Python и API модели

Даёт полный контроль. Подходит для изучения механизма, коротких workflows и систем с особыми требованиями. Требуется самостоятельно реализовать loop, tools, state и обработку ошибок.

OpenAI Agents SDK

Предоставляет компактные primitives: Agent, Runner, tools, guardrails, handoffs, sessions и tracing. Удобен для Python-first разработки и экосистемы OpenAI.

LangChain

Предоставляет abstractions и большое количество интеграций с моделями, tools и хранилищами. Удобен для быстрого соединения компонентов. Большое количество уровней абстракции требует понимания того, что происходит внутри.

LangGraph

Низкоуровневый runtime для stateful и long-running workflows: граф состояния, checkpoints, durable execution и human-in-the-loop. Нужен, когда маршрут имеет ветвления, паузы и возобновление.

LangSmith и альтернативы

Платформы observability и evaluation помогают сохранять traces, запускать эксперименты и сравнивать версии. Можно использовать облачные продукты или строить часть инфраструктуры самостоятельно.

МСП

МСП — протокол интеграций, а не конкурент Agents SDK или LangGraph. Агент, написанный на разных runtime, может использовать одни MCP servers.

Как выбирать

Спроси:

- нужен ли явный граф состояния;
- должна ли задача переживать перезапуск;
- сколько поставщиков моделей используется;
- нужны ли готовые интеграции;
- какие требования к tracing;
- насколько важен контроль над каждым шагом;
- сможет ли команда отлаживать выбранный уровень абстракции.

Самопроверка

- Почему LangGraph не нужен для каждого чат-бота?
- В чём учебная ценность собственного agent loop?
- Почему МСП нельзя сравнивать с LangChain как два одинаковых фреймворка?

Глава 14. SoloGPT и «Колли» как полигон

Мы не знаем их закрытую внутреннюю архитектуру и не пытаемся копировать продукты. Мы используем публично понятные бизнес-процессы для проектирования учебных агентов.

Возможные агенты для SoloGPT

- агент продаж и квалификации;
- агент поддержки по базе знаний;
- агент проверки актуальности знаний;
- QA-агент для диалогов;
- агент анализа причин неуспешной конверсии;
- агент, предлагающий prompt для A/B-теста;
- агент синхронизации и обогащения CRM;
- внутренний помощник менеджера.

Возможные агенты для «Колли»

- голосовой агент повторяемого сценария;
- агент извлечения данных из транскрипта;
- QA-агент звонков;
- агент поиска причин недозвона;
- агент контроля сценария;
- агент подготовки следующего CRM-действия;
- агент предложения улучшений сценария;
- агент мониторинга интеграций.

Один пример: QA-агент диалогов

Вход:

- транскрипт;
- цель сценария;
- политика компании;
- ожидаемые поля.

Tools:

- получить данные сделки;
- проверить обязательные поля;
- найти правило в базе знаний;
- создать замечание для ответственного.

Результат:

- оценка по критериям;
- найденные нарушения;
- доказательства из транскрипта;
- предложение улучшения;
- необходимость человеческой проверки.

Evals:

- совпадение с оценкой эксперта;
- корректность цитат;
- отсутствие выдуманных нарушений;
- стабильность на разных сегментах;
- стоимость анализа.

Второй пример: Customer Success Agent

Он получает CRM, биллинг, подписку, обращения и заметки. Обычный Python рассчитывает деньги и даты. RAG ищет контекст. Агент выбирает разрешённое действие. CRM tool создаёт задачу. Менеджер оценивает рекомендацию. Результат попадает в eval-набор.

Эти два агента отличаются входом, tools, рисками и evals, но могут использовать общий harness.

Самопроверка

- Придумай три разных агента для одной компании.
- Какая часть harness будет общей?
- Какой агент может нанести наибольший ущерб и какие ограничения ему нужны?

Глава 15. Как проектировать нового агента

Перед написанием кода заполни короткий паспорт.

1. Бизнес-результат

Не «использовать LLM», а, например, уменьшить время разбора обращений или повысить долю корректно заполненных CRM-карточек.

2. Почему нужен агент

Можно ли решить задачу обычным кодом, поиском или фиксированным workflow? Агент оправдан, если путь заранее трудно определить и требуется выбор действий по ситуации.

3. Вход и источники истины

Какие данные приходят? Какие источники считаются достоверными? Что делать при конфликте?

4. Tools и права

Какие действия доступны? Какие только читают? Какие меняют данные? Где требуется подтверждение?

5. State и память

Что нужно хранить во время запуска и между запусками? Как долго? Можно ли удалить?

6. Критерий завершения

Как система доказывает выполнение? Когда останавливается? Когда делает handoff?

7. Ошибки и восстановление

Что произойдёт при timeout, неправильном ответе, повторе события или частичном выполнении?

8. Evals

Какие примеры являются хорошими? Какие пограничными? Как проверить tool calls, траекторию, безопасность, стоимость и latency?

9. Observability

Какие traces, версии и метрики нужно сохранить, чтобы воспроизвести ошибку?

10. Релиз

Как новая версия сравнивается с baseline? Кто одобряет? Как откатиться?

Переносимый принцип:

Сначала спроектируй доказательство качества и границы агента, затем выбирай модель и фреймворк.

Глава 16. План чтения до 1 августа

День 1

Главы 1–2. Понять роль AI Engineer и ограничения LLM.

День 2

Глава 3. Structured Outputs и Tool Calling.

День 3

Глава 4. Workflow, agent loop и уровни автономности.

День 4

Глава 5. Agent Harness. Нарисовать harness любого знакомого бизнес-процесса.

День 5

Глава 6. State, memory и context engineering.

День 6

Глава 7. RAG. Придумать ошибки базы знаний.

День 7

Повторить главы 1–7 и написать своими словами: «Как работает агент».

День 8

Глава 8. MCP и интеграции.

День 9

Глава 9. Planning, reflection и multi-agent.

День 10

Глава 10. Evals и observability.

День 11

Глава 11. Self-improving loops.

День 12

Глава 12. Production, безопасность и стоимость.

День 13

Глава 13. Карта фреймворков. Не учить API, а понять различия.

День 14

Глава 14. Придумать по три агента для SoloGPT и «Колли».

День 15

Глава 15. Заполнить паспорт одного будущего агента.

Перед возвращением

Ответить без книги:

1. Что делает систему агентом?
2. Из чего состоит harness?
3. Чем state отличается от memory и context?
4. Зачем нужны tools и MCP?
5. Как оценивать траекторию агента?
6. Когда нужен human-in-the-loop?
7. Почему multi-agent не всегда лучше?
8. Как безопасно устроить self-improving loop?

9. Какие production-ошибки нужно предусмотреть?
10. Как выбрать между чистым Python, Agents SDK, LangChain и LangGraph?

Краткий словарь

Agent — система, где модель динамически выбирает шаги и tools для достижения цели.

Agent Loop — повторяющийся цикл решения, действия, наблюдения и следующего решения.

Agent Harness — инженерная среда вокруг модели: tools, контекст, state, ограничения, выполнение, traces и evals.

API — договор, по которому программы обмениваются запросами и ответами.

Checkpoint — сохранённый снимок состояния долгой задачи.

Context — информация, доступная модели в текущем вызове.

Context Engineering — проектирование содержания и структуры контекста.

Embedding — числовое представление смысла текста.

Eval — систематическая проверка качества AI-системы на наборе сценариев.

Function Calling / Tool Calling — механизм, позволяющий модели запросить выполнение функции приложением.

Guardrail — проверка или ограничение входа, выхода либо действия агента.

Handoff — передача задачи человеку или другому агенту.

Human-in-the-loop — участие человека в проверке, изменении или подтверждении шага.

Idempotency — свойство, при котором повтор запроса не создаёт повторного эффекта.

LLM — большая языковая модель.

LLM-as-Judge — использование модели для оценки результата по заданным критериям.

Memory — сведения, сохраняемые для будущих шагов или запусков.

MCP — стандарт подключения tools, resources и prompts к AI-приложениям.

Multi-agent — система из нескольких взаимодействующих агентов.

Observability — возможность понять состояние и поведение системы по traces, логам и метрикам.

Prompt Injection — попытка внедрить вредоносную инструкцию через входные данные или документы.

RAG — получение релевантных документов перед генерацией ответа.

Reflection / Self-critique — шаг, на котором модель анализирует собственный результат.

Runtime — код, который управляет выполнением агента.

Sandbox — изолированная среда с ограниченными возможностями.

State — структурированное текущее состояние процесса.

Stopping Condition — условие, после которого агент обязан завершить или приостановить работу.

Structured Output — ответ модели по строгой схеме.

Tool — функция или возможность, через которую агент получает данные или совершает действие.

Trace — полная запись шагов одного запуска агента.

Workflow — заранее заданная кодом последовательность шагов.

Источники для дальнейшего чтения

- [Anthropic: Building effective agents](#)
- [Anthropic: Effective harnesses for long-running agents](#)
- [Anthropic: Demystifying evals for AI agents](#)
- [Anthropic: How we built our multi-agent research system](#)
- [Anthropic / Warp: self-improving agents on Claude](#)
- [OpenAI Agents SDK](#)
- [LangChain agents](#)
- [LangGraph overview](#)
- [LangSmith evaluation](#)
- [MCP architecture](#)

Материалы SoloGPT и «Колли» разобраны отдельно в `docs/PRODUCT_REFERENCE.md`.